

План занятия

- Линейная регрессия:
 - при чём здесь переопределённые системы
 - линейная регрессия с МНК
 - линейная регрессия с SGD
- Масштабирование признаков
- Проблема мультиколлинеарности
- Переобучение и регуляризация
- Практика для помощи в ДЗ

Пишите вопросы в чат в любое время. Если хотите спросить голосом – не стесняйтесь, поднимайте руку

Линейная регрессия

Вспоминаем переопределённые системы

Переопределённая система:

- кол-во л.н. уравнений $>$ кол-во переменных
- ранг матрицы системы $>$ кол-во переменных
- не имеет точных решений!
- можно решить приближённо (например, через МНК или рассмотреть как оптимизационную задачу)

Как можно более точное решение переопределённых систем — и есть ML!

- Признаки — матрица системы.
- Таргет — вектор свободных членов.
- Коэффициенты (веса) — неизвестные.

Вспоминаем переопределённые системы

	name	math_score	reading_score	hours_study	final_grade
0	Alice	78	82	10	85
1	Bob	65	70	6	70
2	Charlie	90	88	15	92
3	Diana	72	75	8	78
4	Ethan	85	80	12	88

$$\begin{cases} 85 = w_0 + 78w_1 + 82w_2 + 10w_3 & (\text{Alice}) \\ 70 = w_0 + 65w_1 + 70w_2 + 6w_3 & (\text{Bob}) \\ 92 = w_0 + 90w_1 + 88w_2 + 15w_3 & (\text{Charlie}) \\ 78 = w_0 + 72w_1 + 75w_2 + 8w_3 & (\text{Diana}) \\ 88 = w_0 + 85w_1 + 80w_2 + 12w_3 & (\text{Ethan}) \end{cases}$$

Есть **датасет** для обучения ML-модели: на основе оценок по чтению, математике и часам обучения предсказать финальную оценку студентов.

Запишем задачу в виде **системы линейных уравнений**.

Уравнений > неизвестных $\Rightarrow$ решений у системы нет.

Но можно найти **приближённое** $w = (w_0, w_1, w_2, w_3)$, которое минимизирует ошибку!

Пример переопределённой СЛУ

Задача: найти наилучшее приближение для w_1, w_2 , если

$$\begin{cases} w_1 + w_2 = 1 \\ w_1 + 2w_2 = 2 \\ w_1 + w_2 = 12 \end{cases}$$

Решение: на языке линейной алгебры попытка выразить вектор $(1, 2, 12)^T$ через $(1, 1, 1)^T, (1, 2, 1)^T$ не будет успешной, т.к. они линейно независимы.

$$\underbrace{\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 1 \end{pmatrix}}_A \cdot \underbrace{\begin{pmatrix} w \\ w \end{pmatrix}}_w = \underbrace{\begin{pmatrix} 1 \\ 2 \\ 12 \end{pmatrix}}_b$$

Пример переопределённой СЛУ

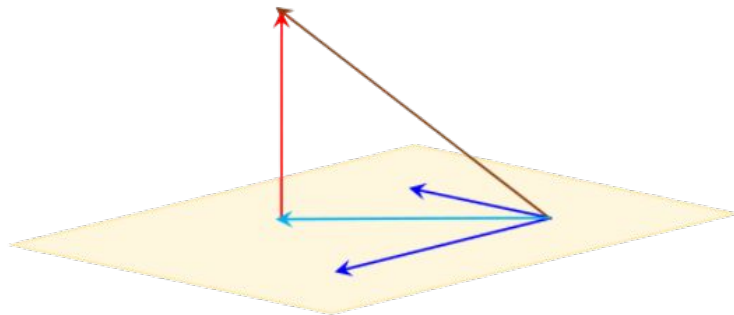
Задача: найти наилучшее приближение для w_1, w_2 , если

$$\begin{cases} w_1 + w_2 = 1 \\ w_1 + 2w_2 = 2 \\ w_1 + w_2 = 12 \end{cases}$$

Решение: на языке линейной алгебры попытка выразить вектор $(1, 2, 12)^T$ через $(1, 1, 1)^T, (1, 2, 1)^T$ не будет успешной, т.к. они линейно независимы.

$$\underbrace{\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 1 \end{pmatrix}}_A \cdot \underbrace{\begin{pmatrix} w \\ w \end{pmatrix}}_w = \underbrace{\begin{pmatrix} 1 \\ 2 \\ 12 \end{pmatrix}}_b$$

Геометрически это значит, что **вектор свободных коэффициентов b** не лежит в одной плоскости со **столбцами матрицы A** .



Пример переопределённой СЛУ

Задача: найти наилучшее приближение для w_1, w_2 , если

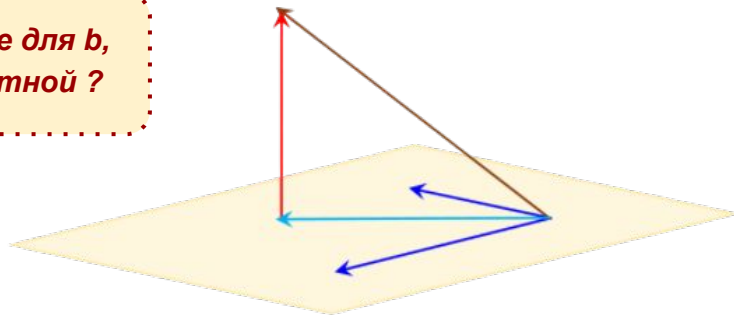
$$\begin{cases} w_1 + w_2 = 1 \\ w_1 + 2w_2 = 2 \\ w_1 + w_2 = 12 \end{cases}$$

Решение: на языке линейной алгебры попытка выразить вектор $(1, 2, 12)^T$ через $(1, 1, 1)^T, (1, 2, 1)^T$ не будет успешной, т.к. они линейно независимы.

$$\underbrace{\begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 1 \end{pmatrix}}_A \cdot \underbrace{\begin{pmatrix} w \\ w \end{pmatrix}}_w = \underbrace{\begin{pmatrix} 1 \\ 2 \\ 12 \end{pmatrix}}_b$$

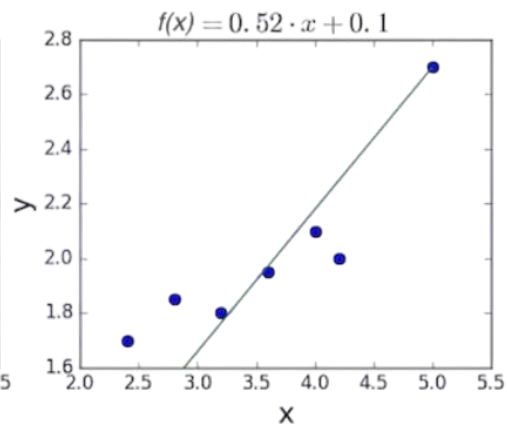
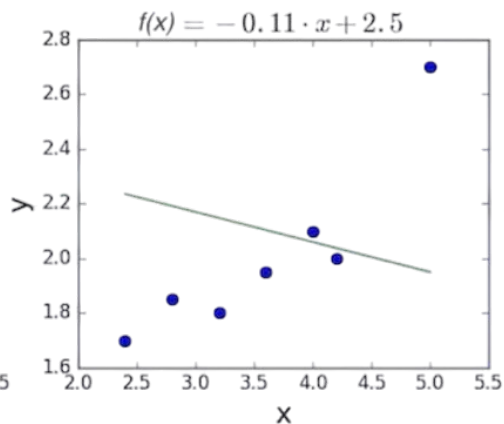
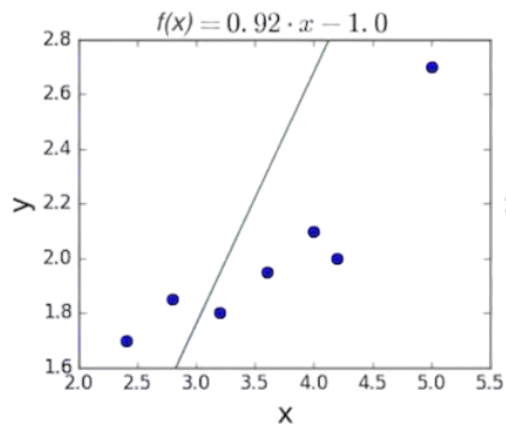
Какое наилучшее приближение для b , чтобы система была совместной ?

Геометрически это значит, что **вектор свободных коэффициентов b** не лежит в одной плоскости со **столбцами матрицы A** .



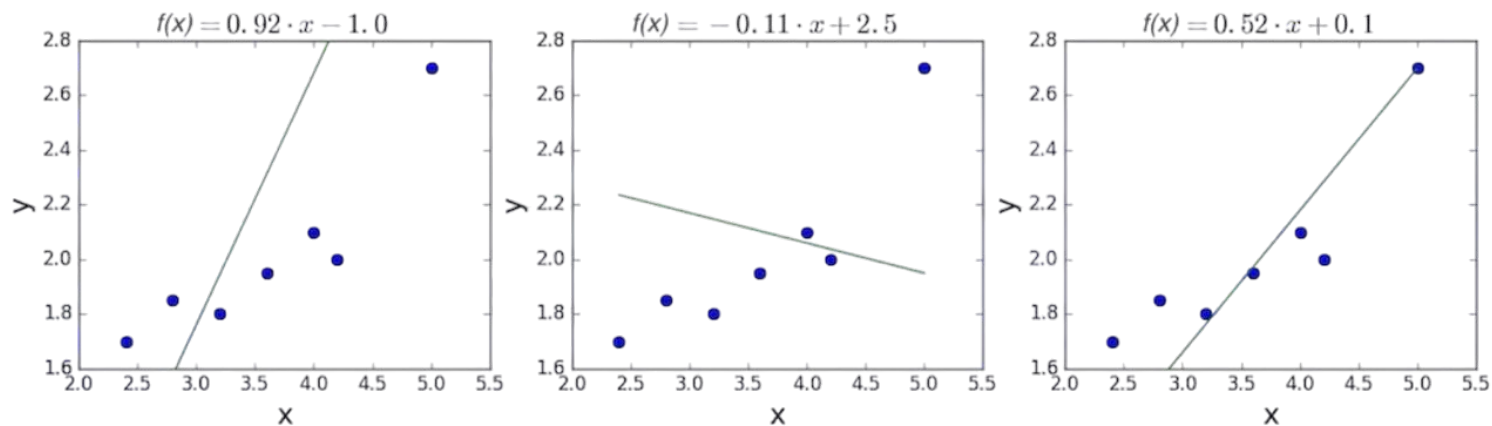
Пример в виде линейной регрессии

Для 1-го признака: какая прямая будет наилучшим приближением?



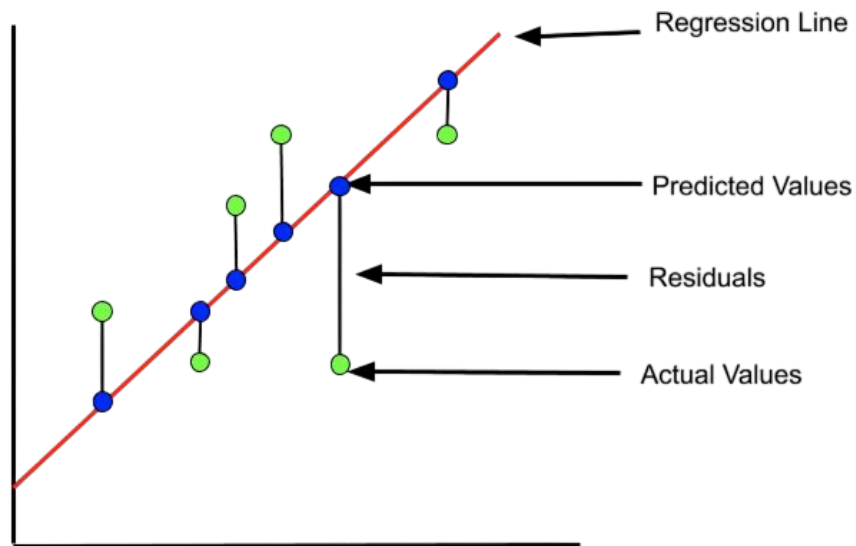
Пример в виде линейной регрессии

Для 1-го признаков: какая прямая будет наилучшим приближением?



Ключевая идея: лучшее решение соответствует прямой, для которой *сумма квадратов отклонений предсказанных значений от истинных минимальна*.

Пример в виде линейной регрессии



Представим входные данные в виде СЛУ
 $Ax = b$, где

- A – матрица признаков (размера $m \times n$, где $m > n$),
- x – вектор коэффициентов, который нужно найти ($n \times 1$),
- b – вектор целевых значений ($m \times 1$).

Метод наименьших квадратов

Математически эта идея реализована в МНК.

Для переопределённой несовместной СЛУ $Ax = b$:

1. ищем такой вектор x , который минимизирует сумму квадратов отклонений $\|Ax - b\|^2$
2. функция ошибки – это *сумма квадратов разностей между реальными и предсказанными значениями*:

$$L(x) = \sum_{i=1}^m (A_i x - b_i)^2 = \|Ax - b\|^2$$

3. чтобы минимизировать $L(x)$, найдем его градиент по x и приравняем к нулю:

$$\nabla_x \|Ax - b\|^2 = 0$$

4. если расписать уравнение выше, получится $A^T Ax = A^T b$
5. выражаем отсюда ответ $x = (A^T A)^{-1} A^T b$

А градиентный спуск?

1. шаги 1-2 из МНК полностью аналогичны, хотим минимизировать:

$$L(x) = \sum_{i=1}^m (A_i x - b_i)^2 = \|Ax - b\|^2$$

2. если рассчитать градиент, он будет следующий:

$$\nabla_x L(x) = \frac{2}{m} A^T (Ax - b)$$

3. вместо аналитического решения будем итеративно обновлять шаг:

$$x_{k+1} = x_k - \alpha \cdot \frac{2}{m} A^T (Ax_k - b)$$

4. повторяем до сходимости (пока обновлять параметры x до тех пор, пока изменения не станут достаточно маленькими – т.е. пока модель почти не перестает улучшаться).

Что на питоне?

```
1 from sklearn.linear_model import LinearRegression
2
3 # Инициализация модели
4 model = LinearRegression()
5
6 # Обучение (методом МНК)
7 model.fit(X, y)
8
9 # Предсказание
10 y_pred = model.predict(X)
```

Линейная регрессия с МНК



Линейная регрессия с
градиентным спуском



```
1 from sklearn.linear_model import SGDRegressor
2
3 # Инициализация модели
4 model = SGDRegressor(
5     learning_rate='constant',
6     eta0=0.01,
7     max_iter=1000,
8     tol=1e-6
9 )
10
11 # Обучение (градиентный спуск)
12 model.fit(X, y)
13
14 # Предсказание
15 y_pred = model.predict([[6]])
```

Масштабирование признаков

Проблема разного масштаба признаков

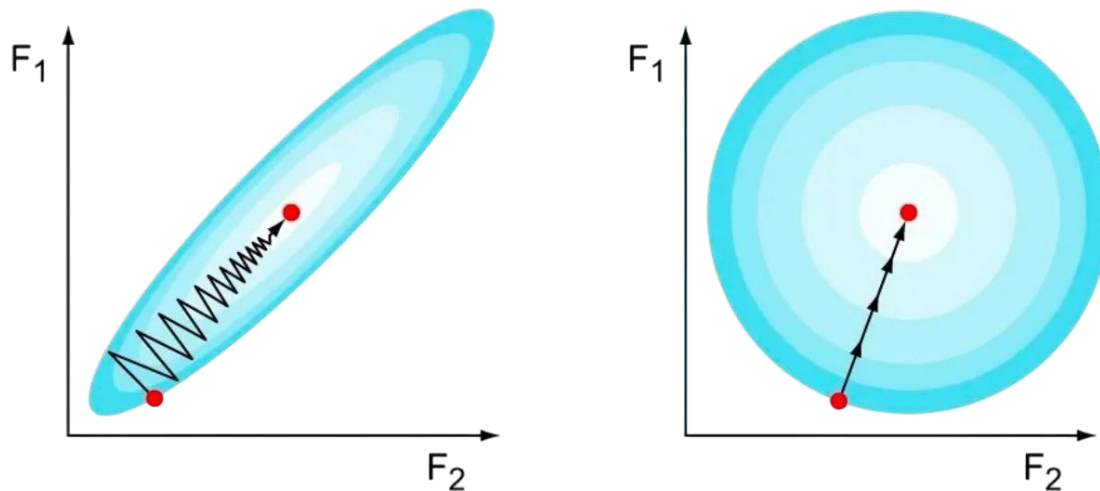
При работе с ML-моделями данные могут содержать признаки с *разными единицами измерения и порядками величин*. Например:

- Возраст: 25, 40, 60 (диапазон десятков)
- Зарплата: 30 000, 60 000, 100 000 (диапазон тысяч)
- Площадь дома: 50, 100, 200 (диапазон сотен)

Это создает проблемы для алгоритмов, особенно использующих метрики расстояния (например, метод ближайших соседей, градиентный спуск).

Проблема разного масштаба признаков

Наглядно проблема показана на рисунке: слева процесс сходимости градиентного спуска на исходных данных (немасштабированных), а справа – на масштабированных.



Как итог: масштабирование улучшает сходимость алгоритмов $\Rightarrow$ повышает качество. + это критически важно при регуляризации и поиске расстояний между векторами.

Виды масштабирования: стандартизация

Формула преобразования:

$$X' = \frac{X - \mu}{\sigma}$$

- μ – среднее значение признака,
- σ – стандартное отклонение.

Когда использовать?

- Если данные похожи на нормальное распределение (колоколообразная форма).

```
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Виды масштабирования: нормализация

Формула преобразования:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

```
from sklearn.preprocessing import MinMaxScaler  
  
scaler = MinMaxScaler()  
X_scaled = scaler.fit_transform(X)
```

- переводит значения признака в диапазон [0,1]

Когда использовать?

- Когда распределение данных далеко от нормального.
- Полезно, если нет выбросов (т.к. выбросы сильно влияют на $X_{\max}$ и $X_{\min}$).

Виды масштабирования: логарифмирование

Формула преобразования:

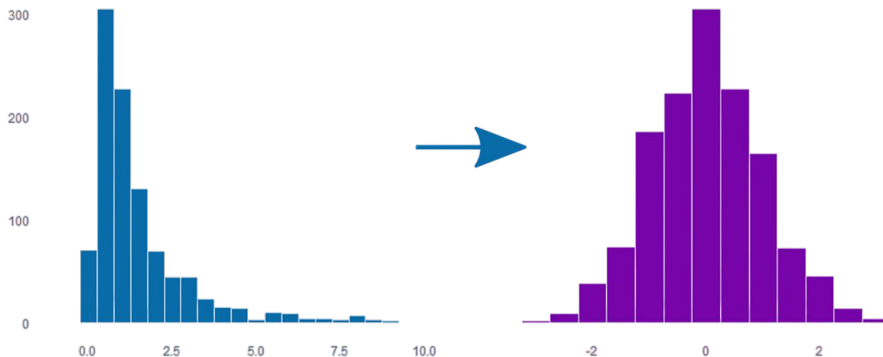
$$X' = \log(X + 1)$$

- добавляем +1, чтобы избежать проблем с $\log(0)$

```
import numpy as np  
  
X_log = np.log1p(X)  #  $\log(1 + X)$ 
```

Когда использовать?

- Если распределение данных сильно скошено.
- Если есть выбросы: логарифмирование их сглаживает.
- Подходит для данных, где разброс значений велик (например, зарплаты, число покупок).



Проблема мультиколлинеарности

Корреляция и мультиколлинеарность

Корреляция – это мера статистической зависимости между двумя или более переменными.

Она показывает, *насколько изменения в одной переменной связаны с изменениями в другой*.

В контексте машинного обучения, корреляция помогает понять, как признаки взаимодействуют между собой.

Мультиколлинеарность – это ситуация, когда один или несколько признаков в данных сильно *коррелируют* друг с другом.

Это может привести к *неустойчивости модели*, поскольку модель не может правильно оценить влияние отдельных признаков, если они содержат много схожей информации.

С точки зрения линейной алгебры матрица системы (признаков) становится *плохо обусловленной* (есть признаки, которые около-линейно-зависимы).

Виды корреляций: Пирсона

Формула преобразования:

$$\rho(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2 \sum_{i=1}^n (Y_i - \bar{Y})^2}}$$

```
# data – pandas.DataFrame  
data.corr(method='pearson')|
```

- X_i и Y_i – значения переменных,
- $\bar{X}$ и $\bar{Y}$ с чертой – средние значения переменных.

Как интерпретировать?

- $\rho = 1$: идеальная положительная линейная зависимость $y=x$.
- $\rho = -1$: идеальная отрицательная линейная зависимость $y=-x$.
- $\rho = 0$: нет линейной зависимости.
- **Итог:** корреляция Пирсона измеряет линейную зависимость 2-х переменных!

Виды корреляций: Спирмена

Формула преобразования:

$$\rho_s = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}$$

```
# data – pandas.DataFrame  
data.corr(method='spearman')
```

- d_i – разница между рангами каждой пары значений для X и Y,
- n – количество наблюдений.

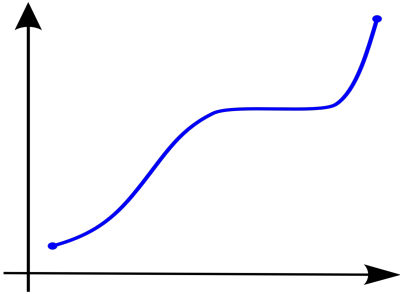
Как интерпретировать?

- $\rho_s = 1$: идеальная монотонная зависимость (вверх).
- $\rho_s = -1$: идеальная монотонная зависимость (вниз).
- $\rho_s = 0$: нет монотонной зависимости.
- **Итог:** корреляция Спирмена измеряет монотонную зависимость 2-х переменных!

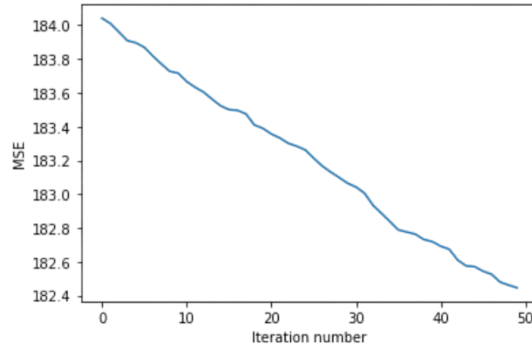
Небольшая викторина!

Напишите для каждой картинки, чему будет равна (примерно) соответствующая корреляция?

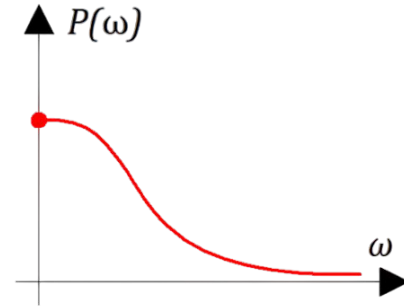
1. Корр. Спирмена:



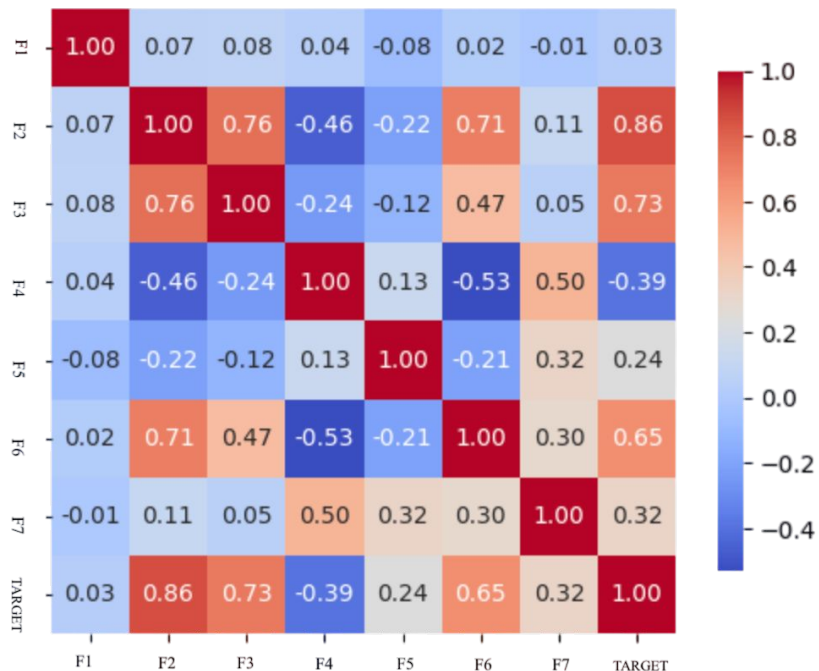
2. Корр. Пирсона:



3. Корр. Спирмена:



Гайд, какие признаки удалить



1. Первым делом находим очень высокие – близкие к 1, или очень низкие – близкие к -1, значения корреляции среди ПРИЗНАКОВ (порогом можно считать >0.7 или <-0.7).
2. Смотрим, у кого из пары выше корреляция с ТАРГЕТОМ, его и оставляем:)

Переобучение и регуляризация

Что такое переобучение

Переобучение (overfitting) – это ситуация, когда модель слишком хорошо запоминает тренировочные данные, но плохо обобщает их на новые данные (зазубривает). В линейной регрессии это проявляется в слишком больших коэффициентах модели, которые подгоняют модель под шум в данных.

Признаки переобучения:

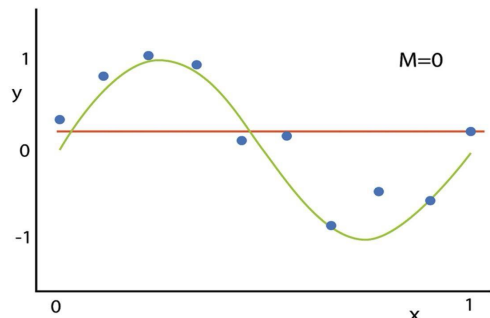
- Низкая ошибка на обучающих данных, но высокая на тестовых.
- Высокие коэффициенты (веса) модели.

Чем может быть вызвано:

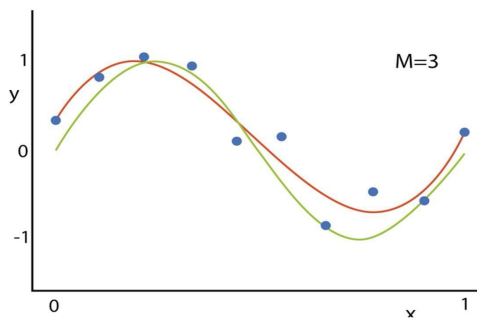
- Мультиколлинеарность.
- Слишком сложная модель.

Переобучение из-за “сложной” модели

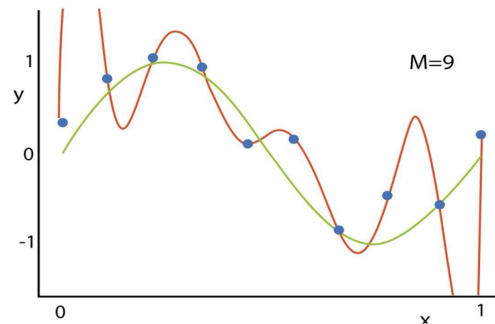
Пример: в качестве модели можно взять *полиномиальную регрессию* – та, которая “пытается аппроксимировать” зависимость не прямой, а полиномом заданной степени. Если переборщить со степенью полинома, может случиться переобучение:)



Недообучение



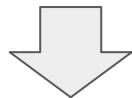
Оптимально!



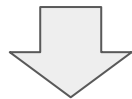
Переобучение

Переобучение из-за мультиколлинеарности

Когда признаки сильно коррелируют, матрица $X^T X$ становится плохо обусловленной, что ведет к нестабильности в решении линейной системы.



Это приводит к большим коэффициентам регрессии, которые зависят от малейших изменений данных.



Модель начинает запоминать шум вместо выявления общих закономерностей.

Пример: если у нас 2 сильно коррелированных признака (например, длина и площадь комнаты), модель может назначить им огромные по модулю, но разнонаправленные коэффициенты. Это делает модель нестабильной и увеличивает вероятность переобучения.

Как бороться с переобучением?

Регуляризация добавляет штраф к функции потерь, заставляя модель тем самым уменьшать масштабы коэффициентов линейной регрессии.

Виды регуляризации:

- L_1 , добавляем к loss $\lambda \sum_{j=1}^n w_j^2$ (плюс – она всюду дифференцируема, штраф сильнее)
- L_2 , добавляем к loss $\lambda \sum_{j=1}^n |w_j|$ (минус – недифференцируема, зато обнуляет веса)
- $L_1 + L_2$ (Elastic Net)

```
from sklearn.linear_model import Lasso # L1
from sklearn.linear_model import Ridge # L2
from sklearn.linear_model import ElasticNet # L1+ L2
```

Спасибо за внимание!